

Structured output and safe inputs

Week 6 Lecture 2: Browser Use: How LLM Agents Drive the Web

Summer 2026

What we will cover

- `output_model_schema`: from prose `final_result` to validated Pydantic data
- How the framework injects the schema into the task and parses the agent's done action
- Accessing structured output: `final_result()`, `structured_output`, `model_validate_json()`
- `sensitive_data`: flat format, domain-scoped format, and the placeholder/secret-tag protocol
- `allowed_domains` on `BrowserProfile` and the security implications of omitting it

By the end, you will be able to run an agent that returns a typed Python object and one that fills in credentials without exposing them to the LLM.

output_model_schema

The problem with prose output

By default, the agent ends by calling the done action with a plain text string.

```
history = await agent.run()  
result = history.final_result()    # str | None
```

This works for simple tasks. But when you need structured data (a list of records, a price, a status), parsing prose is fragile. The text format can drift between runs, model versions, or prompts.

output_model_schema gives the agent a Pydantic model to fill in instead of free text.

Defining an output model

```
from pydantic import BaseModel
from browser_use import Agent, ChatOpenAI

class Post(BaseModel):
    post_title: str
    post_url: str
    num_comments: int
    hours_since_post: int

class Posts(BaseModel):
    posts: list[Post]
```

Pass it to the Agent:

```
agent = Agent(
    task='Go to hackernews show hn and give me the first 5 posts',
    llm=ChatOpenAI(model='gpt-4.1-mini'),
```

What the framework does with the schema

When `output_model_schema` is set, the framework:

1. Serializes the schema to JSON and appends it to the task string
2. Instructs the agent's done action to output JSON conforming to that schema
3. Validates the final result string against the Pydantic model

The LLM is told the expected output format at the start of every run; it is not something the LLM guesses.

Source: `browser_use/agent/service.py`, `_enhance_task_with_schema` (browser-use contributors, 2026)

Reading the result

```
history = await agent.run()

# Raw JSON string (the done action's output)
raw = history.final_result()

# Parse manually
parsed: Posts = Posts.model_validate_json(raw)

# Or use the history property (requires output_model_schema was set on the agent)
parsed: Posts | None = history.structured_output

for post in parsed.posts:
    print(post.post_title, post.num_comments)
```

`structured_output` calls `model_validate_json(final_result())` for you and returns `None` if there is no result.

Source: `browser_use/agent/views.py` `AgentHistoryList.structured_output` (browser-use

sensitive_data

The problem: secrets in the model context

A naive approach passes credentials directly in the task string:

```
# Do NOT do this
agent = Agent(task='Log into example.com with username admin and password s3cr3t', llm=llm)
```

The password is now in every message the LLM processes. It appears in logs, in the conversation history, and potentially in any page the agent visits if the agent copies it to a form and the page content is re-read.

sensitive_data keeps the real value out of the LLM's context entirely.

Flat format: global placeholders

```
sensitive_data = {  
    'x_name': 'my_actual_username',  
    'x_password': 'my_actual_password',  
}  
  
agent = Agent(  
    task='Log into x.com using x_name and x_password',  
    llm=llm,  
    sensitive_data=sensitive_data,  
)
```

The LLM sees only the placeholder names `x_name` and `x_password`. When it needs to type a value, it emits `<secret>x_name</secret>` in the action. The framework substitutes the real value before the browser receives it.

Source: `browser_use/agent/message_manager/service.py` (browser-use contributors, 2026)

Domain-scoped format: credentials per domain

```
company_creds = {  
    'telephone': '9123456789',  
    'email': 'user@example.com',  
    'name': 'John Doe',  
}  
  
sensitive_data = {  
    'httpbin.org': company_creds,  
    'https://*.example-staging.com': company_creds,  
}  
  
agent = Agent(task='Fill in the form at httpbin.org/forms/post', llm=llm, sensitive_data=sensitive_data)
```

Placeholders are exposed only when the agent is on a matching domain. If the agent navigates away, those placeholders disappear from the context.

Source: `examples/features/sensitive_data.py` (browser-use contributors, 2026)

The tag protocol

Inside `_get_sensitive_data_description`, the framework tells the LLM:

IMPORTANT: When entering sensitive values, you MUST wrap the placeholder name in `<secret>` tags.

Example: `<secret>x_name</secret>`

The system will automatically replace these tags with the actual secret values.

The substitution happens in `_filter_sensitive_data`, which scans every outbound message for real secret values and redacts them, replacing them with `<secret>placeholder</secret>`.

This means even if a screenshot contains a password field the agent has filled, the redaction pass scrubs the real value from what the LLM reads back.

Source: `browser_use/agent/message_manager/service.py` (browser-use contributors, 2026)

allowed_domains on BrowserProfile

Locking the browser to specific sites

```
from browser_use import Agent, ChatOpenAI
from browser_use.browser import BrowserProfile, BrowserSession

browser_session = BrowserSession(
    browser_profile=BrowserProfile(
        allowed_domains=['google.com', '*.google.com'],
    )
)

agent = Agent(task='...', llm=ChatOpenAI(model='gpt-4.1-mini'), browser_session=browser_session)
```

The BrowserProfile's `allowed_domains` list restricts where the browser can navigate. Any attempt to go outside these domains is blocked at the browser layer before the agent acts.

Source: `examples/features/restrict_urls.py` (browser-use contributors, 2026)

Why allowed_domains matters with sensitive_data

If you pass sensitive_data without locking allowed_domains, the framework logs a warning:

```
⚠ Agent(sensitive_data=●●●●●●●) was provided but Browser(allowed_domains=[...]) is not locked down!  
💀 If the agent visits a malicious website and encounters a prompt-injection attack, your sensitive_data may be exposed!
```

A prompt-injection attack embeds text on a page that instructs the agent to navigate somewhere else and enter credentials. Without allowed_domains, nothing stops the agent from complying.

Use sensitive_data and allowed_domains together, always.

Source: `browser_use/agent/service.py` (browser-use contributors, 2026)

Take-aways

1. `output_model_schema` turns prose `final_result` into a validated Pydantic object. The schema is injected into the task automatically.
2. Access the result via `history.final_result()` for the raw JSON or `history.structured_output` for a parsed instance.
3. `sensitive_data` keeps real secrets out of the LLM's context. The LLM sees placeholder names; the framework substitutes real values via the tag protocol.
4. Domain-scoped `sensitive_data` (the dict-of-dict format) exposes credentials only on matching domains.
5. Always pair `sensitive_data` with `BrowserProfile(allowed_domains=[...])` to defend against prompt injection.

"The actual username, password, and 2FA codes are filled in programmatically. The agent never sees your actual credentials."

What is next

- **Section (this week):** Build an agent with one custom extraction tool and a structured output model; verify the schema holds
- **Week 7:** Persistent browser profiles, real-browser login, multi-tab work, and downloads
- **Reading (required before section):** Week 6 reading at `/c/browser-use-26su/readings/wk06`